

# Society Analysis System

Changyuan Yu  
Columbia University  
New York, NY, USA  
cy2812@columbia.edu

Zhiliang Wang  
Columbia University  
New York, NY, USA  
zw3162@columbia.edu

Yihang Sun  
Columbia University  
New York, NY, USA  
ys3978@columbia.edu

## Abstract

We present the *Society Analysis System*, a retrieval-augmented analysis platform that lets a non-technical user pose natural-language questions about public discourse and receive citation-bearing reports that cross-check Reddit community discussions against authoritative news sources (AP, Reuters, BBC, NYT). The system fans every question out, in parallel, to three heterogeneous retrieval branches — hybrid evidence retrieval over a curated news corpus, natural-language-to-SQL over a structured pgvector warehouse with 1 536-dim claim embeddings, and Cypher / NetworkX analytics over a Kuzu property graph — under a bounded planner that decides on a per-question basis which branches to call. A four-axis quality critic gates every report, and failed verdicts are routed back into three Chroma-backed memory stores so that the planner, the NL2SQL prompt, and the report writer all learn from their past mistakes. We evaluate the deployed system across nine modules with 530 ground-truth cases, each verified to align with the live data; the system passes every configured target, with NL2SQL Pass@1 of 0.99, RAG Recall@5 of 1.00, planner route F1 of 0.99, and a critic-based hallucination rate of 0.02.

**Keywords:** retrieval-augmented generation, knowledge graphs, NL2SQL, self-reflection, social media analysis, fact-checking

Code: [https://github.com/Sapphirine/2026\\_Society\\_2](https://github.com/Sapphirine/2026_Society_2) •  
Demo: <https://youtu.be/wZ3efwD22Qg>

## 1 Introduction and Problem Formulation

A subreddit is a self-organising social group: users post, reply, and react along reply trees that, in aggregate across topics, form the local society environment we want to characterise. An analyst or moderator looking at this society routinely asks questions that mix several modes of inquiry — *what is this community talking about?*, *how does it feel about that?*, *which accounts are amplifying it?*, *how is the story spreading reply-by-reply?*, and *is the shared claim supported by authoritative reporting?* The *Society Analysis System* answers such free-form questions with one citation-bearing report per turn, by triangulating Reddit community discussions against authoritative news from AP, Reuters, BBC, and NYT, surfacing trend detection, per-topic emotion baselining, echo-chamber and amplifier-account detection, propagation tracing, and claim verification with inline citations to help break rumours [6, 20].

No single retrieval mechanism handles this menu of questions. Dense vector search excels at semantic recall over reportage but degrades on counts and aggregations; structured queries handle counts but cannot match a paraphrased claim against an article body; graph traversal exposes propagation and influence but is brittle when the user’s intent is purely factual. The methodological core question is therefore not *which* retrieval method is best, but rather: *given a free-form society-analysis question, how do we decide which combination of methods to invoke, ground every assertion in verifiable evidence, and detect when the system has hallucinated?*

We contribute a working end-to-end system that operationalises this analysis: (i) *topic-aware question routing* via a bounded multi-branch planner that splits a user message into 1–3 atomic subtasks under deterministic safety bounds ( $\leq 3$  parallel branches per subtask,  $\leq 5$  total per workflow); (ii) *claim-level rumour cross-check against authoritative news*, backed by a dedicated claims table that stores each atomic claim once with a 1 536-dim pgvector embedding, a 64-bit Charikar simhash, and a full-text-search vector, enabling reciprocal-rank-fusion retrieval that resolves the “fact-check returns zero rows” failure mode of plain full-text search; (iii) *hallucination-aware reporting* via a four-axis quality critic that programmatically validates citation completeness and numeric consistency and uses a separate LLM pass for on-topic and hallucination checks, with one bounded retry before surfacing a needs-human-review flag to the moderator; (iv) *continual learning from analyst-visible failures* via a self-curating reflection store that routes critic verdicts to three Chroma collections so the rewriter, planner, and NL2SQL prompts see past route violations, empty-result SQL patterns, and citation failures as negative few-shot; and (v) a *nine-module evaluation harness* with 530 ground-truth cases whose chunk ids, topic ids, entity names, and SQL row counts were each verified to exist in the running snapshot before being committed. The system is implemented in Python on FastAPI and Streamlit, runs as three Docker containers, and ships a frozen 2 072-post / 20-topic / 62-claim demonstration snapshot of one subreddit’s recent activity so that the evaluation can be reproduced without re-ingestion.

## 2 Related Work and Background

**Retrieval-augmented generation.** Classical RAG [11] combines dense retrieval [10] with BM25 [16] via reciprocal rank fusion [7], optionally followed by a cross-encoder rerank [21]. Our novelty is the retriever’s *placement*: it is one

of three branches selected by an upstream planner, so RAG is invoked only when the question is grounded in reportage.

**Text-to-SQL.** The Spider [23] / BIRD [12] benchmarks and DIN-SQL [15]’s decomposed prompting set the template for our NL2SQL branch (memory-augmented few-shot with a  $\leq 3$ -round repair loop). We add a schema-meta sync that mirrors per-column descriptions from Postgres into Chroma, and a negative-example store of failed SQL patterns.

**Knowledge graphs and propagation.** PageRank [14] and Louvain [3] remain the canonical primitives for influence and community detection on social graphs. Empirical work on news diffusion [20] shows that misinformation propagates faster and farther than verified news — the phenomenon our cascade and propagation queries are designed to surface. We use Kuzu [8] as the property-graph engine and lift the induced topic subgraph into NetworkX for analytics.

**Society analysis platforms.** A line of empirical work in computational social science has quantified specific society-level pathologies of online platforms: echo-chamber formation across topics and platforms [6], the limits of cross-cutting exposure on Facebook [2], and the measurable influence of fake news during election cycles [4]. In parallel, several public platforms operationalise individual society-analysis tasks: Hoaxy and the Observatory on Social Media track the diffusion of low-credibility content via social bots [17], Botometer / BotometerLite score accounts for automated / amplifier-like behaviour [22], and ClaimBuster identifies check-worthy claims for downstream fact-checking pipelines [9]. Each prior platform addresses *one* sub-task in isolation. Our contribution is to expose these signals — topic structure, echo-chamber score, amplifier ranking, propagation paths, and claim-level cross-checks against authoritative news — behind a single natural-language query interface, so that an analyst can ask one question (“*what are the dominant claims in this topic, who is amplifying them, and do official sources back them up?*”) and receive a citation-bearing, hallucination-gated answer that uses several of these detectors at once.

**Self-reflection and fact-checking.** Reflexion [18], Self-Refine [13], and Self-RAG [1] introduce the pattern of critiquing and revising LLM output. Our quality critic is in this family but deliberately split: the cheap axes (citation completeness, numeric consistency to within  $10^{-6}$ ) are programmatic, and only the expensive axes (on-topic, hallucination) call an LLM. Unlike FEVER [19], where claims are given against a fixed corpus, our claims are extracted from Reddit submission titles at ingest time, and the verification output is a structured per-claim audit rather than a single SUPPORTED/REFUTED label.

## 3 System Design and Methodology

### 3.1 Multi-agent architecture

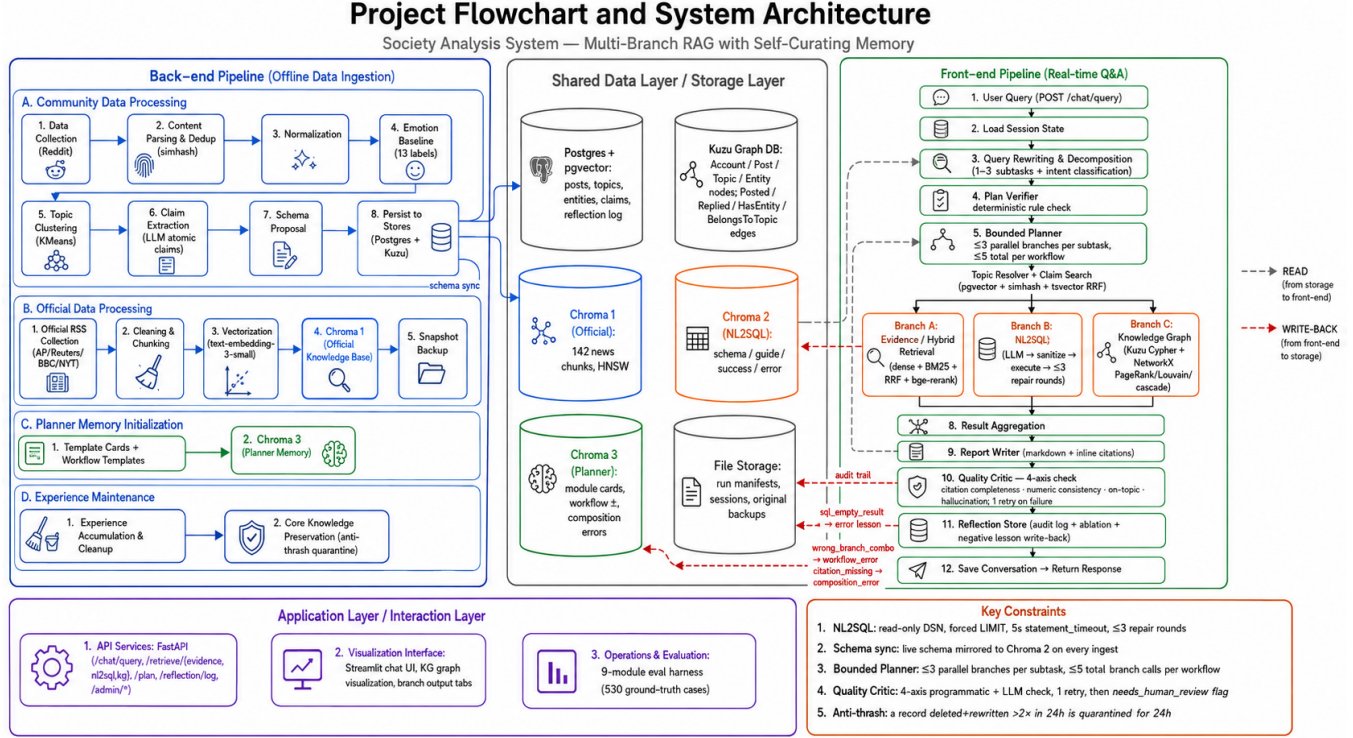
The online question-answering surface is structured as a cooperative *multi-agent* system rather than a monolithic prompt. A single **ChatOrchestrator** sequences the request through nine specialised agents that communicate via typed Pydantic contracts: **QueryRewriter** decomposes the user message into 1–3 Subtasks with intent labels; **PlanVerifier** rule-checks the rewrite; **BoundedPlannerV2** resolves topic / claim hints and dispatches each subtask to retrieval-branch agents under hard fan-out bounds; the three branch agents **EvidenceAgent**, **NL2SQLAgent**, and **KGAgent** run hybrid RAG, memory-augmented SQL, and Cypher / NetworkX analytics in parallel; **ReportWriter** composes a citation-bearing markdown answer; **QualityCritic** gates the answer on four axes; and **ReflectionStore** persists the turn and writes negative lessons back into the shared memories. The orchestration is a directed acyclic flow with one asynchronous back-edge (the reflection write, which only affects *later* turns), so every fan-out is bounded ( $\leq 3$  branch agents per subtask,  $\leq 5$  branch calls per workflow) and every inter-agent boundary is independently testable. The three Chroma collections (Sec. 4) act as the shared blackboard.

### 3.2 End-to-end chat flow

Figure 1 shows the full architecture of the system, spanning the offline ingestion pipeline, the shared data and storage layer, the online multi-agent flow, the planner-memory initialisation, and the application surface. Every chat-query request flows through the agents listed above. The QueryRewriter produces a strict-JSON rewritten query whose intents are drawn from eleven labels: *fact check*, *topic-claim audit*, *community listing*, *trend*, *propagation trace*, *influencer query*, *coordination check*, *community structure*, *cascade query*, *comparison*, and *free-form*. The PlanVerifier applies deterministic rules and writes any correction to the planner memory (Chroma 3) as a route-violation anti-pattern.

### 3.3 The planner agent

**BoundedPlannerV2** is the orchestration agent that decides which retrieval-branch agents fire for each subtask. For each subtask, it (i) resolves user phrases like “the virus topic” to a topic id via a TopicResolver that runs a cosine search against pre-embedded topic centroids; (ii) for fact-check or topic-claim audit intents, runs the dedicated ClaimSearchTool (described below) to pre-resolve claim-id hints; (iii) scrubs official-source mentions out of the natural-language query handed to the NL2SQLAgent (otherwise the LLM tries to filter the posts table by “AP”, which has no meaning in the community-side schema); and (iv) dispatches the chosen branch agents in parallel under the bounds noted above. The planner pulls *module-capability cards* (declaring when each



**Figure 1.** End-to-end pipeline and system architecture. Top-left: the offline ingestion pipeline. Centre: the shared data layer (Postgres, Kuzu, three Chroma collections, file store). Right: the online multi-agent flow with up to three parallel retrieval branches feeding the critic-and-reflection loop. Bottom: the application surface. Solid arrows are data flow; dashed arrows are memory feeds and reflection writes.

branch agent is and is not appropriate) and past workflow-success / workflow-error pairs from the planner memory, both as positive and negative few-shot.

### 3.4 Three retrieval-branch agents

The three branch agents share a common shape — pre-filter, candidate generation, fusion, optional rerank, structured output — and differ in the underlying primitives.

**EvidenceAgent (RAG branch).** A six-step pipeline against the official-news collection: metadata pre-filter, dense recall with the OpenAI text-embedding-3-small model (cosine), BM25 recall over the filtered subset (rebuilt per query), reciprocal rank fusion at  $k=60$ , optional cross-encoder rerank with bge-reranker-base [21], and citation extraction (title, outlet, URL, publish date). For society-analysis purposes, this branch provides *official corroboration* of any community claim.

**NL2SQLAgent (text-to-SQL branch).** The agent embeds the natural-language query, recalls schema, guide, success, and error records from the NL2SQL memory, and builds a prompt that includes the pre-resolved topic-id and claim-id hints from the planner. The LLM emits a single SELECT,

which a sanitiser guards (row limit, DML stripped, multi-statement blocked) before execution against a read-only Postgres connection with a short per-statement timeout. On failure, up to three repair rounds run before the failed pattern is persisted as an error lesson. A special path skips the LLM for the topic-claim audit intent: a deterministic SELECT against the claims table joined with the post-claim link table returns canonical claim text. This branch surfaces *aggregate community behaviour* — counts, distributions, and trends over the discussion.

**KGAgent (knowledge-graph branch).** Each subtask intent maps to a Cypher query against Kuzu, with the induced topic subgraph lifted into NetworkX where required: a propagation trace runs a reply-chain traversal between two account ids; an influencer query runs PageRank; a coordination check runs Louvain; a community-structure query computes a modularity-based echo-chamber score; and a cascade query produces a depth-ranked reply tree. Subgraph extraction is LRU-cached by topic id, and writes bump a sequence so that reads see fresh edges. This branch exposes the *relational and propagation structure* of the community — who replies to whom, who amplifies, where the discussion fragments.



### 3.5 Critic and reflection agents

The **QualityCritic** agent validates four axes of every report: *citation completeness* (every inline chunk-id reference resolves to a real chunk) and *numeric consistency* (every number matches a branch value within  $10^{-6}$ ) are checked programmatically; *on-topic* and *hallucination* use one LLM call each. A first-round failure triggers a single retry of the writer with a different sampling temperature; a second failure flips the response’s needs-human-review flag to true.

The **ReflectionStore** agent owns the closed loop. After every turn it (a) writes one row to the reflection log; (b) when the critic verdict failed, runs an ablation that re-executes the failing step with the suspect record removed and, if the failure disappears, deletes the record from its owning Chroma collection (with anti-thrash quarantine after  $> 2$  delete-then-rewrite cycles in 24h); and (c) writes negative lessons back — an empty SQL result becomes an NL2SQL-memory error, a wrong or missing branch combination becomes a planner-memory workflow error, and a citation or numeric mismatch becomes a planner-memory composition error.

## 4 Implementation and Data

### 4.1 Storage stack

The system uses three persistent stores (Table 1). The Postgres schema defines eight tables: a posts table (with maintained full-text-search and simhash columns), a topics table (with a trigger-maintained post-count column), an entities table, a post-entity link table (M:N), a claims table (with a 1536-dim vector column under an HNSW cosine index, plus a 64-bit simhash and a full-text-search vector), a post-claim link table (M:N), a schema-description table (per-column descriptions, mirrored to the NL2SQL memory), and a reflection log. The NL2SQL branch connects through a read-only Postgres connection; all writes are gated through the offline pipeline.

### 4.2 The hybrid claim retriever

A motivating failure of an earlier prototype was that fact-check queries returned zero rows. The reason was structural: Reddit submission titles *are* the claims, but the discussion lives in comments, whose body text does not contain the claim string. A semantic search over comment text therefore systematically failed to recover the claim that the comments were responding to. Our fix is a dedicated claims table populated at ingest time. For a new query, the ClaimSearchTool computes three ranked lists in a single SQL pass:

1. dense cosine similarity on the claim-embedding column (HNSW index);
2. 64-bit Charikar simhash [5] Hamming distance against the query simhash — catches near-duplicates and paraphrases at Hamming  $\leq 3$ ;
3. full-text-search keyword match against the query (Postgres *plainto\_tsquery*).

**Table 1.** Persistent stores and their roles.

| Store                 | Contents                                                                                | Purpose                                             |
|-----------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------|
| PostgreSQL + pgvector | 8 tables incl. posts, claims, reflection log                                            | Structured warehouse, audit trail, claim embeddings |
| Kuzu (embedded)       | Account / Post / Topic / Entity nodes; Posted, Replied, HasEntity, BelongsToTopic edges | Propagation traversal, PageRank, Louvain            |
| Chroma 1 (official)   | 142 news chunks (HNSW)                                                                  | Evidence retrieval target                           |
| Chroma 2 (NL2SQL)     | schema / guide / success / error records                                                | NL2SQL prompt few-shot + negative examples          |
| Chroma 3 (planner)    | module cards / workflow $\pm$ / composition errors                                      | Planner + writer few-shot + anti-patterns           |

The three lists are merged with reciprocal rank fusion at  $k=60$ , and the top- $k$  claim hits are returned to the planner along with the comment posts that discuss each claim (via the post-claim link table). This replaces an LLM-mined claim approximation with a canonical, indexable record.

### 4.3 Offline pipeline, snapshot, and tech stack

The offline ingestion pipeline runs as eight chained stages: (1) *fetch posts* scrapes a Reddit subreddit via praw; (2) *ingest* performs multimodal enrichment, entity extraction, and simhash dedup with Hamming  $\leq 3$  merging; (3) *normalise* canonicalises text and metadata; (4) *emotion baseline* classifies each post into a 13-label emotion taxonomy with gpt-4o; (5) *topic clustering* runs KMeans over OpenAI embeddings, batched at  $\leq 2000$  posts per run; (6) *extract claims* extracts atomic claims from submission titles, simhash-deduplicates them, and links each claim to the discussing comments via the post-claim link table; (7) *schema propose* introspects newly emerged columns and double-writes their descriptions into the schema-description table and the NL2SQL memory; and (8) *persist* commits the Postgres and Kuzu writes within a single transactional run id.

The shipped *demonstration snapshot* contains 2072 Reddit posts (50 link submissions, 2022 comments) clustered into 20 topics, 62 atomic claims with 2594 post-to-claim links, and 142 official news chunks (AP = 20, Reuters = 20, BBC = 62, NYT = 40). Postgres is restored from a compressed snapshot dump on first boot, and the Chroma / Kuzu stores are bind-mounted into the container, so a clean compose-up reproduces every evaluation number without re-ingestion.

The system is implemented in Python with FastAPI, Streamlit, and Pydantic. All LLM calls use gpt-4o and all embeddings use text-embedding-3-small; reranking uses bge-reranker-base

Cold start is a few minutes on first launch (image build plus reranker download); warm restart takes seconds.

## 5 Evaluation and Results

### 5.1 Evaluation harness

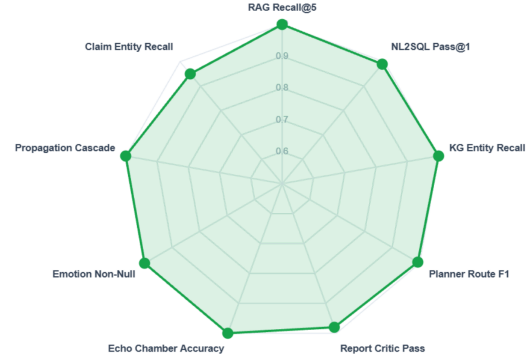
We evaluate the system across nine modules using 530 ground-truth cases. The harness is one module per dimension, sharing the standard ranking metrics (Recall@ $k$ , NDCG@ $k$ , mean reciprocal rank); the output is a JSON summary plus per-module result files and a static HTML dashboard. A first-class concern is *ground-truth alignment*: every chunk id, topic id, account name, expected SQL row count, and entity referenced by an expected answer was verified to exist in the running snapshot before being committed, and every question for the four routing-sensitive modules was sent through the planning endpoint and confirmed to route to the intended branch combination. This precludes the common pitfall of a ground-truth case silently encoding an assumption about model behaviour rather than data presence.

### 5.2 Per-module results

Table 2 reports the measurement against each configured target, and Figure 2 shows the same numbers as a radar plot. All nine modules pass; several — RAG Recall@5, KG entity recall, planner route recall, echo-chamber accuracy, emotion in-set rate, and propagation cascade-found rate — saturate at 1.00. The two metrics between target and ceiling are informative: RAG NDCG@10 of 0.8746 shows that while the top-5 always contains a relevant chunk, the top-10 ranking is imperfect on questions whose relevant chunks come from outlets with very similar lead paragraphs; NL2SQL Result Accuracy of 0.87 is 0.12 below Pass@1, indicating that almost every SQL is syntactically valid but some runs return a row count outside the expected range because the LLM picks a different but defensible filter (e.g. a case-insensitive LIKE on the body text vs. a topic-id join).

## 6 Discussion

The contribution of this system to society analysis is broader than adjudicating individual claims as true or false. By coupling *official corroboration* (RAG), *aggregate community behaviour* (NL2SQL), and *relational and propagation structure* (KG) behind a single natural-language interface, it lets an analyst follow the full life of a topic inside a community: how a conversation forms, which accounts amplify it, whether the group fragments into closed clusters, how the dominant emotion shifts as the topic evolves, and how a claim spreads from an individual post into the group narrative. The headline numbers in Section 5 therefore characterise the reliability of these society-level traces, not just point-wise fact-checks.



**Figure 2.** Per-module evaluation results as a target-vs-actual radar plot across all nine modules. Every axis sits at or above its configured target.

### 6.1 Insights for the analyst

Read through a society-analysis lens, the most informative single number in Table 2 is the writer’s First Pass Rate of 0.95 paired with a Critic Pass Rate of 0.98: an analyst can usually take the report at face value, and the 3% recovered by the retry are exactly the cases where the analyst would otherwise be silently misled. The 0.02 hallucination rate names the surviving error mode — a Reddit comment quoted as if it were an official source — which is exactly the moderator-relevant failure the critic was designed to catch. The planner’s 0.99 route F1 matters because the system cleanly separates the two questions an analyst typically asks together: *what is the community saying* (NL2SQL and KG signals over the community-side data) and *is it true* (RAG against authoritative news). Each branch grounds a different kind of claim, so the analyst can act on each independently — e.g., escalate an echo-chamber finding even when the official-source check returns “no comment.”

### 6.2 Use cases

The capability menu of Section 1 maps directly onto moderator-facing workflows the system already supports on the shipped snapshot. *Trend monitoring* aggregates posts per topic and per recency window. *Emotion baselining* reports the dominant label per topic from the 13-label taxonomy, flagging shifts (e.g. a topic going from neutral to outrage). *Echo-chamber surveillance* runs Louvain on the topic subgraph and reports a modularity-based score, isolating topics where reply behaviour has fragmented into closed clusters. *Amplifier mapping* runs PageRank on the same subgraph to identify accounts whose replies disproportionately drive local visibility, and *propagation tracing* resolves how a story spread

**Table 2.** Per-module evaluation results on the demonstration snapshot ( $n=100$  for the four large modules,  $n=30$  for the four analytic modules, totalling 530 cases). All nine modules pass every configured target. “–” marks diagnostic metrics without a numeric target.

| Module          | Metric              | Target      | Actual | $n$ | Notes                           |
|-----------------|---------------------|-------------|--------|-----|---------------------------------|
| RAG / Evidence  | Recall@5            | $\geq 0.92$ | 1.0000 | 100 | Hybrid + RRF + rerank           |
|                 | NDCG@10             | $\geq 0.90$ | 0.8746 | 100 | Imperfect inside top-10         |
|                 | MRR                 | $\geq 0.85$ | 0.9758 | 100 | First hit nearly always rank-1  |
| NL2SQL          | Pass@1              | $\geq 0.95$ | 0.9900 | 100 | Up to 3 repair rounds           |
|                 | Result Accuracy     | $\geq 0.90$ | 0.8700 | 100 | Filter-style differences        |
| Knowledge Graph | Entity Recall       | $\geq 0.75$ | 1.0000 | 100 | 7 query kinds                   |
|                 | Entity Pass Rate    | $\geq 0.80$ | 1.0000 | 100 |                                 |
| Planner         | Route Recall        | $\geq 0.90$ | 1.0000 | 100 | 5 branch types                  |
|                 | Route F1            | $\geq 0.87$ | 0.9933 | 100 |                                 |
|                 | Exact Match         | –           | 0.9800 | 100 | Diagnostic                      |
| Report Writer   | Critic Pass Rate    | $\geq 0.90$ | 0.9800 | 100 | After up to 1 retry             |
|                 | First Pass Rate     | $\geq 0.75$ | 0.9500 | 100 | No retry needed                 |
|                 | Hallucination Rate  | –           | 0.0200 | 100 | Diagnostic                      |
| Echo Chamber    | API Success         | $\geq 0.95$ | 1.0000 | 30  | 10 topics $\times$ 3 thresholds |
|                 | Classification Acc. | $\geq 0.75$ | 1.0000 | 30  |                                 |
|                 | Modularity In-Range | $\geq 0.90$ | 1.0000 | 30  |                                 |
| Emotion         | SQL Pass            | $\geq 0.95$ | 1.0000 | 30  | 13 emotion labels               |
|                 | Non-Null            | $\geq 0.80$ | 1.0000 | 30  |                                 |
|                 | In-Set              | $\geq 0.75$ | 1.0000 | 30  |                                 |
| Propagation     | API Success         | $\geq 0.95$ | 1.0000 | 30  | cascade + path                  |
|                 | Cascade Found       | $\geq 0.60$ | 1.0000 | 30  |                                 |
|                 | Path API Valid      | $\geq 0.90$ | 1.0000 | 30  |                                 |
| Claim Verify    | API Success         | $\geq 0.90$ | 1.0000 | 30  | 0% fallback                     |
|                 | Key Entity Recall   | $\geq 0.75$ | 0.9500 | 30  |                                 |
|                 | Evidence Found Rate | $\geq 0.80$ | 1.0000 | 30  |                                 |
|                 | Claim Coverage      | –           | 0.8333 | 30  | Diagnostic                      |

reply-by-reply between two accounts. *Rumour interruption* chains the claim search, the RAG cross-check against AP / Reuters / BBC / NYT, and the quality critic into a single per-claim audit that returns a verdict alongside the matched official sentence, so a moderator can attach a citation rather than a ban.

### 6.3 Limitations and threats to validity

Four caveats apply. First, the evaluation is on a single frozen snapshot, so the numbers should be read as an upper bound on behaviour against data the system has already seen through reflection; performance on a freshly ingested, unseen day of Reddit and RSS is not yet measured. Second, the harness excludes the optional end-to-end LLM-as-judge module (Gemini 2.5 Pro) from the default run because of its API cost; its V1 measurement on  $n=30$  was 0.825 answer relevance against a target of 0.80, but a V2 run at  $n=100$  has not yet been completed. Third, the snapshot’s date range is bounded

by what was on Reddit and the four RSS feeds on 2026-05-08, and historical backfill is not implemented, so the system should not be used for time-series questions beyond that window. Fourth, and most important for a society-analysis deployment: the system *surfaces signals; it does not adjudicate* – echo-chamber scores, amplifier rankings, and claim verdicts are designed to be reviewed by a moderator-in-the-loop, who decides whether to escalate, label, or otherwise act, and the 0.02 hallucination rate exists precisely because that human review remains load-bearing. Reproducibility is end-to-end: a clean compose-up plus a single evaluation command reproduces every number in Table 2 in around fifteen minutes.

## References

- [1] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *International Conference on Learning Representations (ICLR)*.

- [2] Eytan Bakshy, Solomon Messing, and Lada A. Adamic. 2015. Exposure to ideologically diverse news and opinion on Facebook. *Science* 348, 6239 (2015), 1130–1132.
- [3] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast Unfolding of Communities in Large Networks. *Journal of Statistical Mechanics: Theory and Experiment* 2008, 10 (2008), P10008.
- [4] Alexandre Bovet and Hernán A. Makse. 2019. Influence of fake news in Twitter during the 2016 US presidential election. *Nature Communications* 10, 1 (2019), 7.
- [5] Moses S. Charikar. 2002. Similarity Estimation Techniques from Round- ing Algorithms. In *Proceedings of the 34th Annual ACM Symposium on Theory of Computing (STOC)*. 380–388.
- [6] Matteo Cinelli, Gianmarco De Francisci Morales, Alessandro Galeazzi, Walter Quattrociocchi, and Michele Starnini. 2021. The echo chamber effect on social media. *Proceedings of the National Academy of Sciences (PNAS)* 118, 9 (2021), e2023301118.
- [7] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. 2009. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 758–759.
- [8] Xiyang Feng, Guodong Jin, Ziyi Chen, Chang Liu, and Semih Salařakova. 2023. Kuzu Graph Database Management System. <https://kuzudb.com>.
- [9] Naeemul Hassan, Fatma Arslan, Chengkai Li, and Mark Tremayne. 2017. Toward Automated Fact-Checking: Detecting Check-worthy Factual Claims by ClaimBuster. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 1803–1812.
- [10] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 6769–6781.
- [11] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*. 9459–9474.
- [12] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2024. Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [13] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [14] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. *The PageRank Citation Ranking: Bringing Order to the Web*. Technical Report 1999-66. Stanford InfoLab.
- [15] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [16] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009), 333–389.
- [17] Chengcheng Shao, Giovanni Luca Ciampaglia, Onur Varol, Kai-Cheng Yang, Alessandro Flammini, and Filippo Menczer. 2018. The spread of low-credibility content by social bots. *Nature Communications* 9, 1 (2018), 4787.
- [18] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [19] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. FEVER: A Large-Scale Dataset for Fact Extraction and VERification. In *Proceedings of NAACL-HLT*. 809–819.
- [20] Soroush Vosoughi, Deb Roy, and Sinan Aral. 2018. The Spread of True and False News Online. *Science* 359, 6380 (2018), 1146–1151.
- [21] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2024. C-Pack: Packed Resources for General Chinese Embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*.
- [22] Kai-Cheng Yang, Onur Varol, Pik-Mai Hui, and Filippo Menczer. 2020. Scalable and Generalizable Social Bot Detection through Data Selection. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [23] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 3911–3921.